

1. First Palindromic String in an Array and find its complexity

Aim

To write a Python program to find the **first palindromic string in an array** and analyze its time and space complexity.

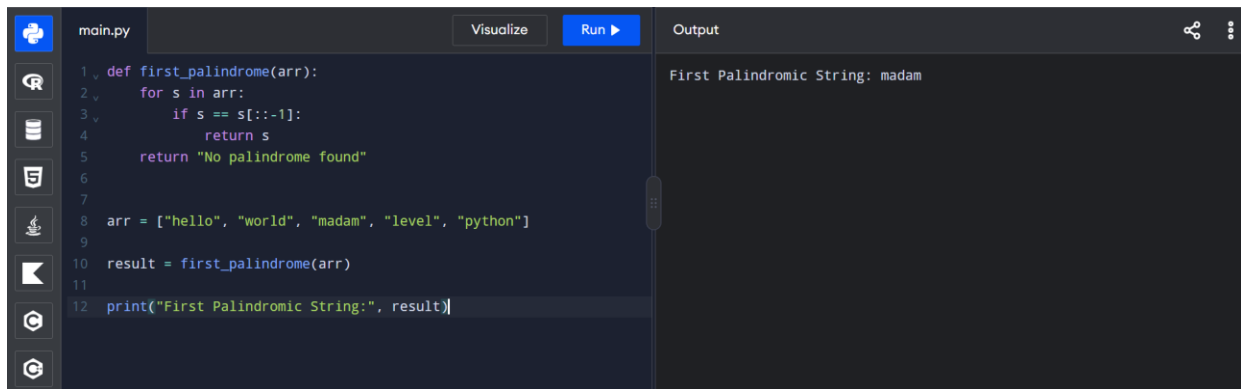
Algorithm

1. Start.
2. Read the array of strings.
3. Traverse the strings one by one.
4. For each string, reverse it and compare it with the original string.
5. If both are equal, print the string as the first palindrome and stop.
6. If no palindrome is found, print "No palindrome found".
7. Stop.

Source Code

```
def first_palindrome(arr):  
    for s in arr:  
        if s == s[::-1]:  
            return s  
    return "No palindrome found"  
  
arr = ["hello", "world", "madam", "level", "python"]  
result = first_palindrome(arr)  
print("First Palindromic String:", result)
```

Output



```
main.py Visualize Run ▶ Output
1 def first_palindrome(arr):
2     for s in arr:
3         if s == s[::-1]:
4             return s
5     return "No palindrome found"
6
7
8 arr = ["hello", "world", "madam", "level", "python"]
9
10 result = first_palindrome(arr)
11
12 print("First Palindromic String:", result)
```

First Palindromic String: madam

Complexity

- **Best Case:** $O(m)$
- **Worst Case:** $O(n \times m)$
- **Space Complexity:** $O(m)$

Where **n** = number of strings and **m** = length of a string.

Result

Thus, the Python program to find the **first palindromic string in an array** was successfully implemented and executed. The first palindromic string was "**madam**". The time complexity is **$O(n \times m)$** in the worst case and the space complexity is **$O(m)$** .